Embedded YOLO from scratch

Tiny-YOLO in NumPy, bit-exact INT8 and an HLS accelerator for the Kria KV260

Léonard Rivals

October 22, 2026

Goal and method

The goal

Usual route: train in a framework, quantize with a vendor tool, map onto a preconfigured unit (Vitis AI DPU). Every layer hides choices: preprocessing, rounding, activation, decoding.

This project takes the opposite route: rebuild every layer by hand, so that every number can be traced to a line of code and checked against the stage before.

- no deep-learning library: NumPy only
- INT8 integer model, bit-exact down to the HLS kernel
- one hand-written engine for Tiny-YOLOv2, Tiny-YOLOv3 and YOLOv4-tiny
- dependability: faults, ABFT, OOD monitor

Workloads

Tiny-YOLOv2 VOC: 15.86 M params, 3.49 GMAC

Tiny-YOLOv3: 8.71 M params, 2.74 GMAC

YOLOv4-tiny: 6.06 M params, 3.45 GMAC

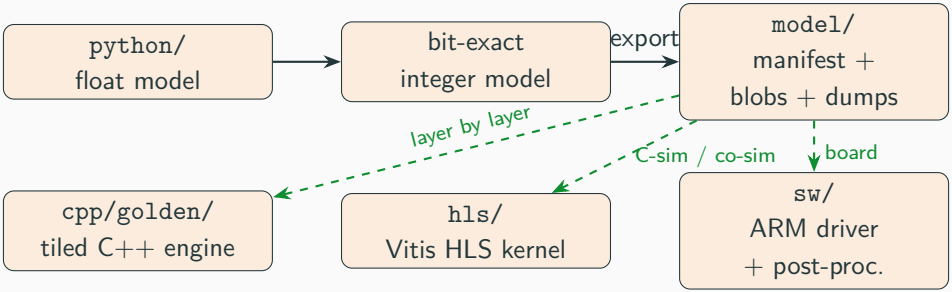
YOLOv7-tiny (in progress): 2.89 GMAC

input 416×416

Target

Kria KV260 (Zynq UltraScale+ XCK26), 200 MHz

The development chain



Each hand-over is checked for **equality of bytes**, not closeness. Specification: `yolo-embarque-de-zero.md`; decisions: 3 ADRs (pure NumPy, HLS, board choice).

Work plan

M0–M3 NumPy building blocks, loss, training, inference, float mAP

M4–M5 INT8 quantization, golden C++

M6–M8 HLS accelerator, SoC, measurements

M9.x HW post-processing, REQ-YOLO, 4 bits, streaming

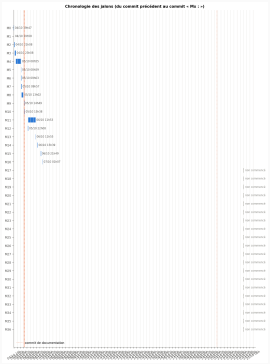
M10 kernel optimisation cascade

M11–M18 datasets, PC profiles, figures, notebooks, Colab campaign, article

M19–M22 English, webcam demo, ASL, auto-labeller

M23–M26 two MACs per DSP, YOLOv4-tiny, dependability, recent models

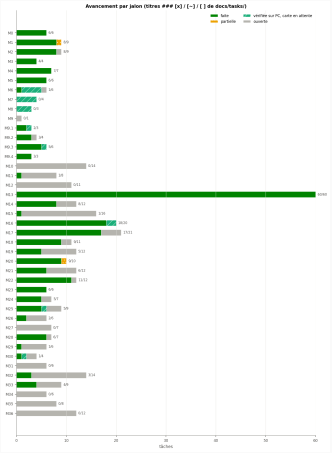
M27–M31 safety (planned)



State at the last render: 231 of 397 tasks done across 41 milestones, 7 milestones complete.

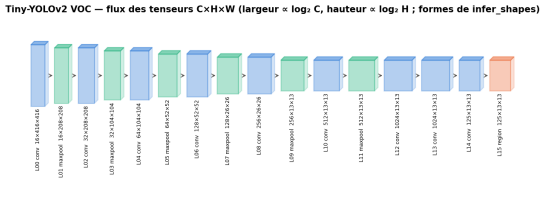
Progress by milestone

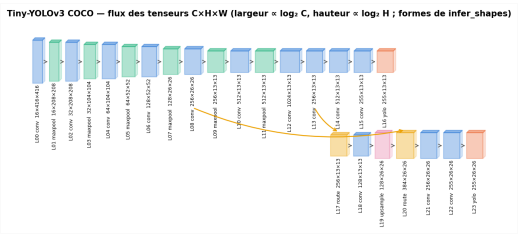
Milestone	Tasks	Done	Progress
M0	6	6	100 %
M1	9	8	89 %
M2	9	8	89 %
M3	4	4	100 %
M4	7	7	100 %
M5	6	6	100 %
M6	6	1	17 %
M7	4	0	0 %
M8	3	0	0 %
M9	1	0	0 %
M9.1	3	2	67 %
M9.2	4	3	75 %
M9.3	6	5	83 %
M9.4	3	3	100 %
M10	14	0	0 %
M12	11	0	0 %
M11	8	1	12 %
M13	60	60	100 %
M14	12	8	67 %
M15	16	1	6 %
M16	20	18	90 %
M17	21	17	81 %
M18	11	9	82 %
M19	12	5	42 %
M20	10	9	90 %
M21	12	6	50 %
M22	12	11	92 %
M23	6	6	100 %
M24	7	5	71 %
M25	9	5	56 %
M26	6	2	33 %
M27	7	0	0 %
M28	7	6	86 %
M29	6	1	17 %
M30	4	1	25 %
M31	6	0	0 %
M32	14	3	21 %
M33	9	4	44 %
M34	6	0	0 %
M35	8	0	0 %
M36	12	0	0 %
Total	**397**	**231**	**58 %**



From scratch in NumPy (M0–M3)

The networks



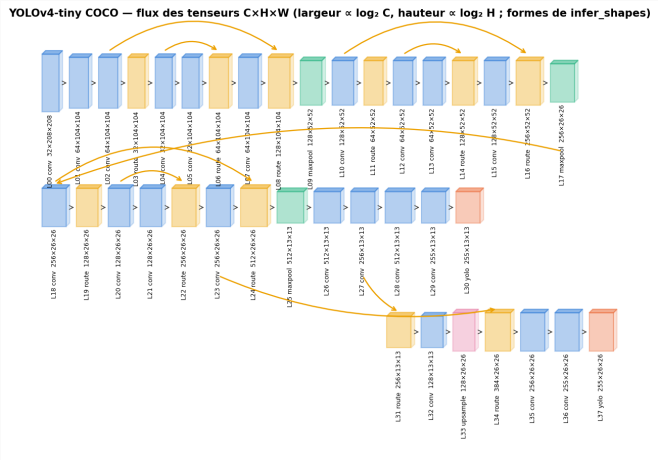


Tiny-YOLOv2: 9 conv 3×3 + BN + leaky ReLU, 6 max-pools, linear 1×1 head with $5 \times (5 + C)$ filters.

Tiny-YOLOv3: a second head at 26×26 fed by upsample + route.

Counted by `make count-macs` from the network descriptions.

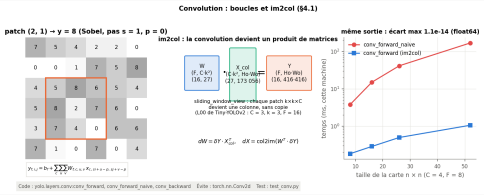
YOLOv4-tiny and YOLOv7-tiny (M24, M26)

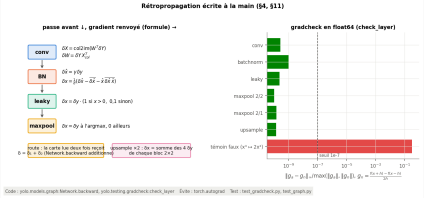


YOLOv4-tiny: CSP blocks; a *grouped route* reads half of the channels of a map (one more addressing mode, no copy); `scale_x_y` on the box centres.

YOLOv7-tiny (in progress): ELAN blocks with 4-source concatenations, SPP with stride-1 pools 5/9/13, exported as chained 5×5 pools. Float, calibration and export done; accuracy not measured yet.

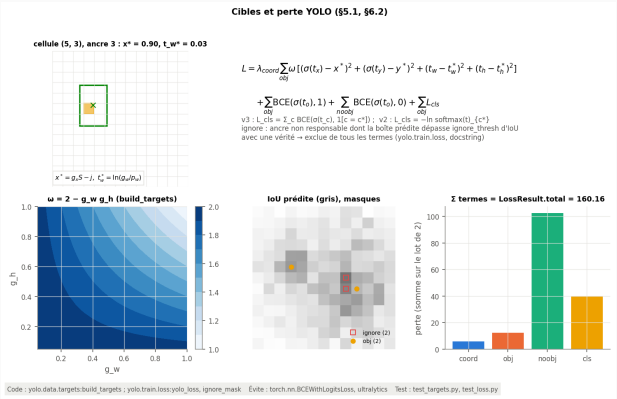
Layers with forward and backward passes





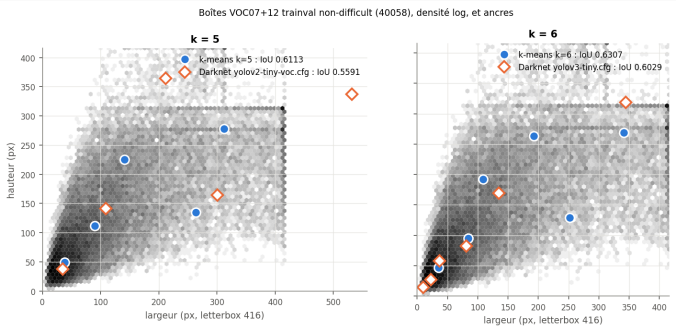
Interface `forward(x) -> (y, cache)`, `backward(dy, cache) -> (dx, grads)`. Naive and im2col convolutions agree to 10^{-12}; every layer passes a float64 finite-difference gradient check (max norm).

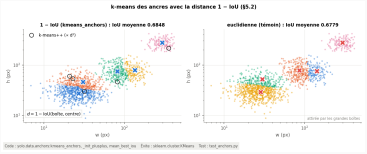
Targets, loss and training



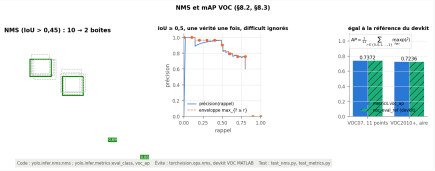
- YOLOv3 loss: box offset regression, logistic objectness with an *ignore* rule, multi-label BCE for the classes
- targets on the best anchor (IoU of shapes); anchors by k-means with $1 - \mathrm{IoU}$
- SGD, burn-in, multi-scale inputs
- **one image overfitted**: targets, loss and optimiser validated together

Anchors from k-means





Inference and VOC evaluation

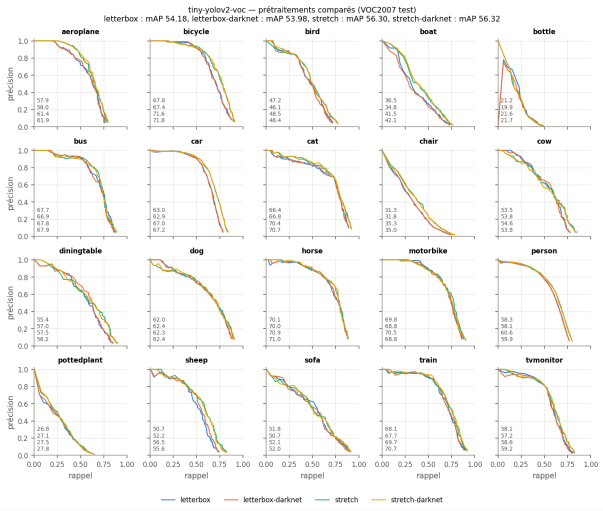


- YOLOv2 decoding, per-class NMS at IoU 0.45
- VOC2007 protocol: 11-point AP, `difficult` ignored; matches the official devkit to 10^{-12}

Preprocessing	Interpolation	mAP
direct resize (`stretch`)	Pillow bilinear	56.30
direct resize (`stretch`)	Darknet (`resize_image`)	56.32
letterbox	Pillow bilinear	54.18
letterbox	Darknet (`resize_image`)	53.98

Float reference with Darknet weights: **56.30** mAP, against $57.1^{\text{pub.}}$ published. Preprocessing matters: a letterbox loses ≈ 2 points with these weights, so the reference uses the direct resize (also the simplest in hardware).

Precision–recall by preprocessing

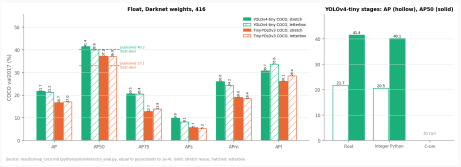


COCO val2017 (M24)

Network	Preprocessing	AP	AP50	AP75	APs	APm	APl
YOLOv4-tiny	stretch	21.7	41.4	20.5	9.9	26.0	30.7
YOLOv4-tiny	letterbox	21.2	39.9	20.4	8.1	24.2	33.6
Tiny-YOLOv3	stretch	16.7	37.2	12.7	5.7	19.0	26.1
Tiny-YOLOv3	letterbox	17.0	36.9	13.8	5.2	18.4	28.4

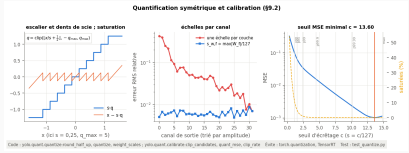
COCO metric, equal to `pycocotools` to 10^{-4}.
YOLOv4-tiny: **41.4** AP50 (published $40.2^{\text{pub.}}$ on test-dev). Tiny-YOLOv3 is optimistic: val2017 overlaps its COCO 2014 training images.

INT8: 40.1 AP50, $-$**1.3** point, above the one-point target; the loss is spread over the backbone, QAT open.



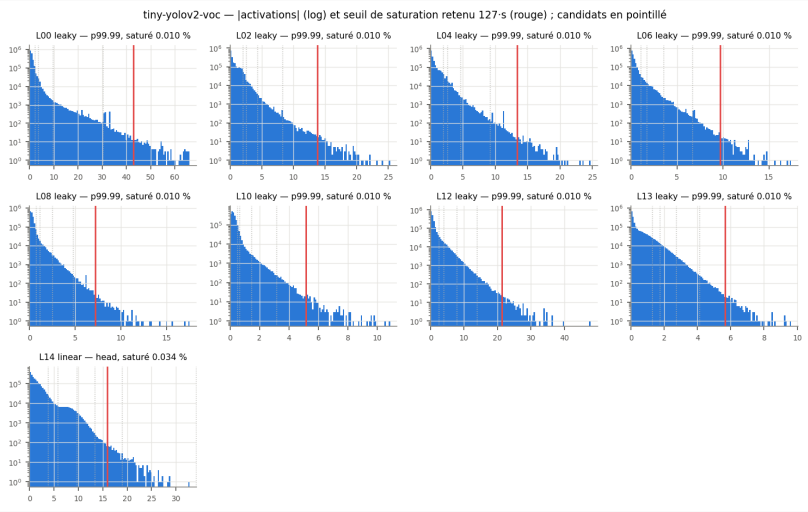
INT8 quantization (M4)

An integer-only network



- BN folded into the weights; weights per output channel, activations per layer ($\mathrm{p99.99}$ of $|x|$ on 500 images, MSE criterion)
- 32-bit accumulator, requantization by an integer multiplier M_0 and a shift: **no floating point left**
- leaky slope $13/128$ as $(13y + 64) \gg 7$ (rounding, not flooring: no bias accumulating across layers); sigmoid and exp by 256-entry tables, step $1/8$

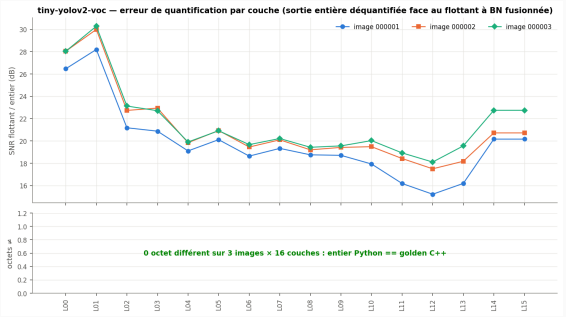
Calibration



Accuracy of the integer model

Preprocessing	Float (fused BN)	Integer INT8	Gap
stretch (`make eval-float`)	56.30	55.66	−0.64
letterbox (the training one)	54.18	53.68	−0.50

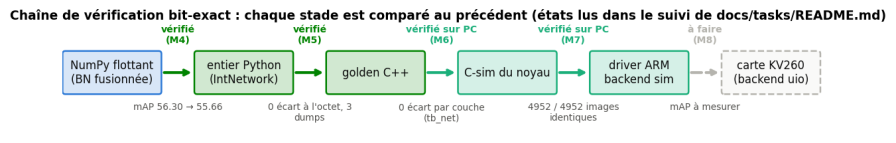
Drop of $-\mathbf{0.64}$ point: below the one-point target, no QAT needed. Most sensitive layer alone: L10$^{\text{tier R}}$ ($-0.61^{\text{tier R}}$), within subset noise: the loss is spread out.



Golden model and verification (M5)

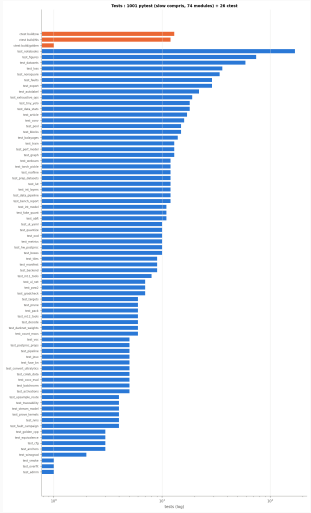
A C++ golden model that runs like the hardware

The quantized network is exported as a manifest + binary blobs, with the integer activations of every layer on a few images (dumps). The C++17 golden model reads the same export and runs it *as the hardware will*: tiled convolution with fixed buffers, bias, requantization, leaky + fused max-pool, route by addressing.



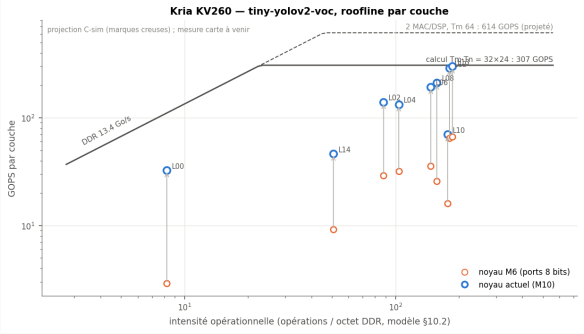
- `make golden-check`: 12 dumps compared layer by layer
- `make csim-gcc`: HLS kernel == golden; `make sw-sim`: ARM driver == golden
- all of it in `make ci` on every commit (synthetic export without weights)

Tests


Tests : 1001 pytest (slow compris, 74 modules) + 24 ctest
test_training
test_gradcheck
test_layers
test_metrics
test_tools
test_activations
test_losses
test_optimizers
test_model
test_serialization
test_integration
test_edge_cases
test_regression
test_utils
test_config
test_cli
test_export
test_quantize
test_profiler
test_benchmark
test_golden
test_fixed_point
test_hls_emulation
test_tiling
test_dataflow
test_resource_model
test_scheduler
test_power
test_pareto
test_dse
test_codegen
test_testbench
test_project
test_cosim
test_golden_model

HLS accelerator and SoC (M6–M7)

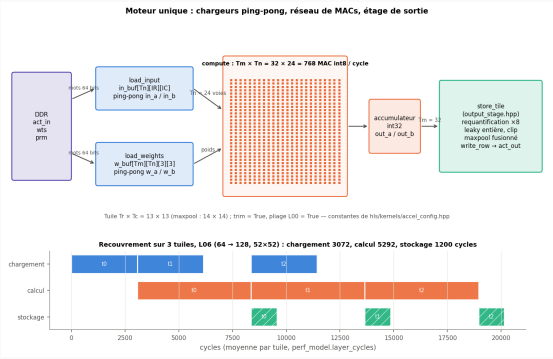
Choosing the board before writing HLS

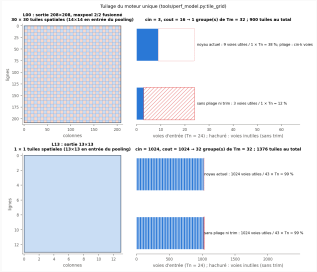


A roofline explorer (`make roofline`) over the candidate boards (PYNQ-Z2, Zybo Z7-20, Ultra96-v2, KV260).

ADR 0003: the KV260 gives about **6**× the roofline performance of a Zynq-7020 board for these networks, at the same price.

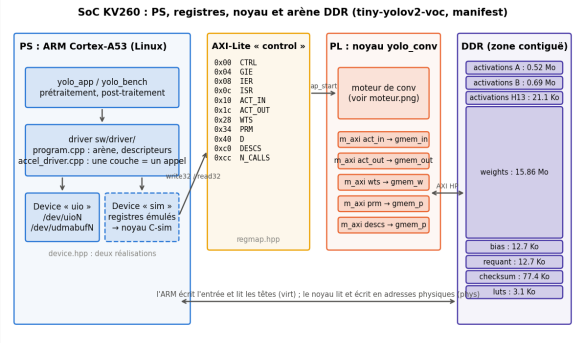
One engine, layer by layer





Loop nest tiled over output/input channels, rows and columns: $T_m = 32$, $T_n = 24$, $T_r = T_c = 13 \Rightarrow$ **768 MAC/cycle** at 200 MHz. Upsample and route cost no compute: they change the read address of the next layer.

SoC integration

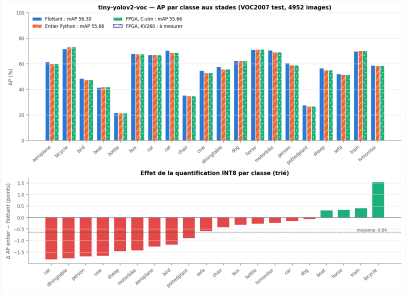


- ARM loads manifest + blobs in contiguous memory, programs the AXI-Lite registers layer after layer, waits for the interrupt
- block design, device-tree overlay and firmware are scripted (`make vivado-build`, `fpga-firmware`)
- **not run yet**: Vivado/Vitis and the board are not available; everything above the hardware runs on the PC through a simulation backend

Results (M8)

mAP at the three stages of the chain

Stage	mAP	Equality with the integer model	Status
Float (fused BN, NumPy float32)	56.30	—	measured
Integer, Python (`IntNetwork`, bit-exact)	55.66	reference	measured
FPGA, C-sim (ARM driver, sim backend, PC)	55.66	4,952 / 4,952 identical images	csim
FPGA, KV260 (uio backend)	—	to be measured	—



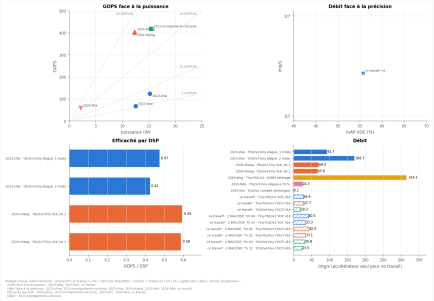
On the **4,952 images** of VOC2007 test, the ARM driver over the C-simulated kernel gives detections **identical** to the Python integer model: the whole accuracy loss comes from quantization.

Projected performance on the KV260 (projection, not measured)

Network	GMAC	Mcycles	ms	img/s	GOPS	MAC efficiency (%)	Status
`tiny-yolov2-voc`	3.486	7.44	37.2	26.9	187.3	61.0	projection
`tiny-yolov3-coco`	2.782	7.21	36.1	27.7	154.3	50.2	projection
`tiny-yolov4-coco`	3.454	10.36	51.8	19.3	133.3	52.3	projection

Tiny-YOLOv2: **37.2$^{\text{proj.}}$ ms**, 26.9$^{\text{proj.}}$ img/s, 187.3$^{\text{proj.}}$ GOPS, efficiency 61.0$^{\text{proj.}}$ %. The cycle model `tools/perf_model.py` equals the C-sim counters cycle for cycle; it is a lower bound (II = 1, no pipeline depth, DDR latency or ARM control).

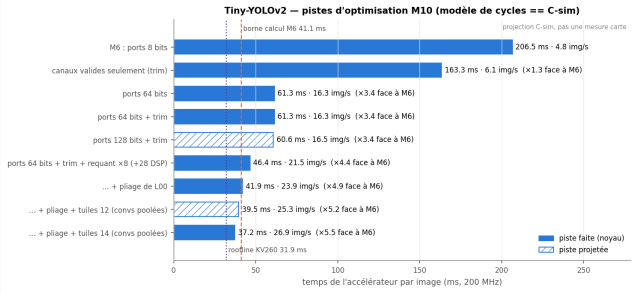
Against the state of the art



- **2024-zhang**, only reference on the same model: two MACs per DSP and HW post-processing; packing (M23) closes the multiplier gap
- same board as **2026-fata** (Vitis AI DPU, 70 % pruning, QAT): the projected kernel exceeds it in img/s *without pruning*

Kernel optimisation (M10)

From 206.5$^{proj.}$ to 37.2$^{proj.}$ ms (projection, not measured)



The M6 kernel spent most of its cycles outside the MAC loop, waiting for its 8-bit ports and its output stage.

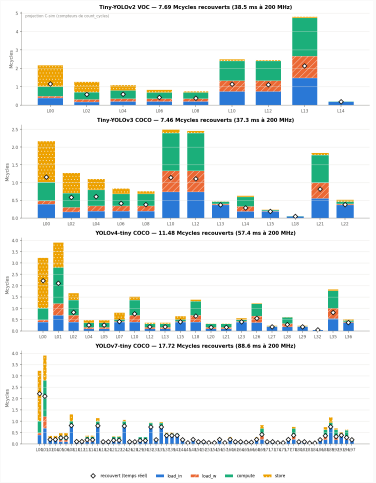
Each step of the cascade stays C-sim equal to the golden model.

The optimisation cascade, step by step (projection, not measured)

Kernel configuration (Tiny-YOLOv2)	Mcycles	ms	img/s	GOPS	MAC efficiency (%)
M6: ports 8 bits	41.30	206.5	4.8	33.8	11.0
valid channels only (trim)	32.67	163.3	6.1	42.7	13.9
ports 64 bits	12.27	61.3	16.3	113.6	37.0
ports 128 bits + trim	12.13	60.6	16.5	115.0	37.4
ports 64 bits + trim + requantization $\times 8$ (+28 DSP)	9.29	46.4	21.5	150.1	48.9
... + L00 folding	8.38	41.9	23.9	166.4	54.2
... + 14$\times$14 tiles for pooled convs (M10 kernel)	7.44	37.2	26.9	187.3	61.0
compute bound of M6 (free loads)	8.23	41.1	24.3	169.5	55.2
compute bound of the M10 kernel	6.32	31.6	31.7	220.7	71.8
KV260 roofline point (roofline.md)	—	31.9	31.3	218.5	—
ideal (MACs / 768, efficiency 100 %)	4.54	22.7	44.1	307.2	100.0

Status of every number of this table: projection.

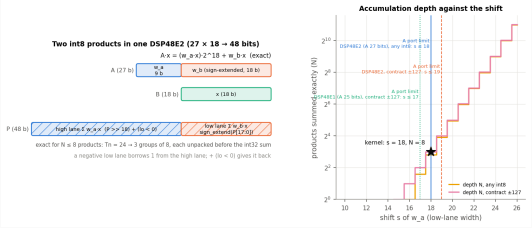
Where the cycles go now



64-bit ports, valid input channels only, 8 channels requantized per cycle, first layer folded onto the input lanes, 14×14 tiles for pooled convolutions.

Compute bound $31.6^{\text{proj.}}$ ms, roofline point $31.9^{\text{proj.}}$ ms.

Two INT8 MACs per DSP (M23)



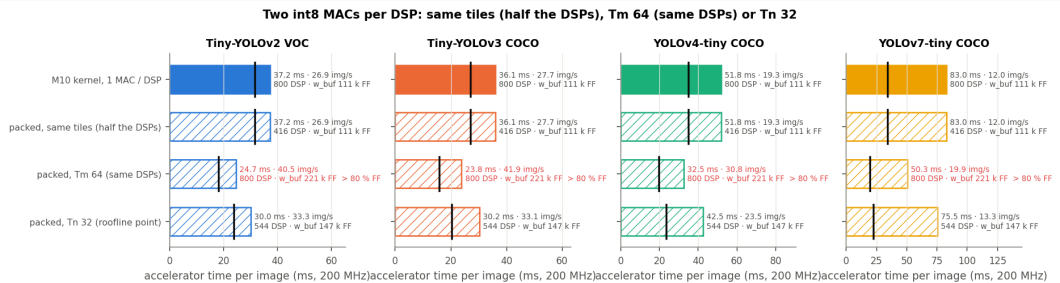
- weights of two output channels in the 27-bit port: $A = w_a 2^{18} + w_b$, one product gives both
- exact for 8 accumulated products under the ± 127 contract: the 24-lane tree is cut into 3×8
- only the multiplication changes: C-sim == golden **to the byte**, cycles unchanged
- same tiles: half the DSPs; $T_m = 64$: $24.7^{\text{proj.}}$ ms (flip-flops over budget); $T_n = 32$: $30.0^{\text{proj.}}$ ms (projection, not measured)

Packed PE options (projection, not measured)

Network	Option	Tm, Tn	PE DSP	ms	GOPS
Tiny-YOLOv2 VOC	M10 kernel, 1 MAC / DSP	32, 24	768	37.2	187.3
Tiny-YOLOv2 VOC	packed, same tiles (half the DSPs)	32, 24	384	37.2	187.3
Tiny-YOLOv2 VOC	packed, Tm 64 (same DSPs)	64, 24	768	24.7	282.5
Tiny-YOLOv2 VOC	packed, Tn 32 (roofline point)	32, 32	512	30.0	232.2
Tiny-YOLOv3 COCO	M10 kernel, 1 MAC / DSP	32, 24	768	36.1	154.3
Tiny-YOLOv3 COCO	packed, same tiles (half the DSPs)	32, 24	384	36.1	154.3
Tiny-YOLOv3 COCO	packed, Tm 64 (same DSPs)	64, 24	768	23.8	233.4
Tiny-YOLOv3 COCO	packed, Tn 32 (roofline point)	32, 32	512	30.2	184.0

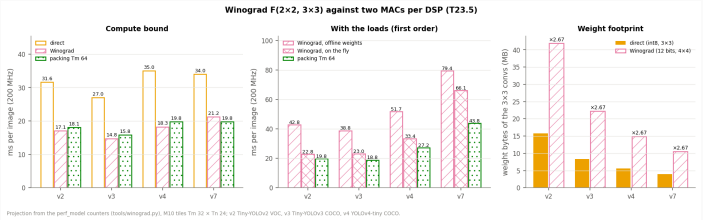
Status of every number of this table: projection.

Packed PE options per network (projection, not measured)



Cycle model == C-sim (tools/perf_model.py:pack_table), projection, not a board measurement. Black tick: compute bound. Hatched: packed PE (ACC_PACK=2). Budget: 80 % of the Kria KV260 DSP and flip-flops (w_buf in registers).

Winograd against packing



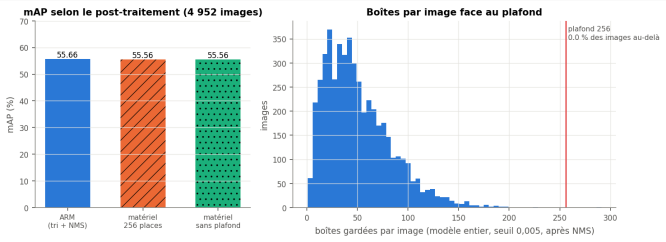
Projection from the perf_model counters (tools/winograd.py), M10 tiles Tm 32 × Tn 24; v2 Tiny-YOLOv2 VOC, v3 Tiny-YOLOv3 COCO, v4 YOLOv4-tiny COCO.

Integer-exact $F(2 \times 2, 3 \times 3)$: fewer multiplications on the 3×3 convs, slightly ahead on the compute bound alone.

With the loads it loses: wider transformed weights consumed faster, a weight buffer beyond the flip-flops of the board, and products that do not pack two to a DSP. **Studied, not implemented.**

Research extensions (M9)

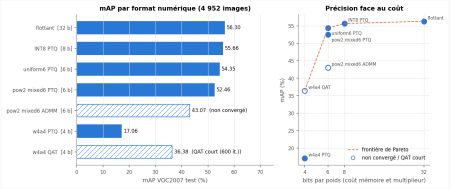
Hardware post-processing (M9.1)



- a second HLS kernel decodes the heads and runs the NMS **in integers only**: threshold on the integer logit, one reciprocal per cell, Q4 pixel corners, IoU as an integer inequality, 256 slots in stream order, no sort
- 55.56 mAP; equal to its golden mirror on 1,014$^{\text{C-sim}}$ cases; synthesis pending

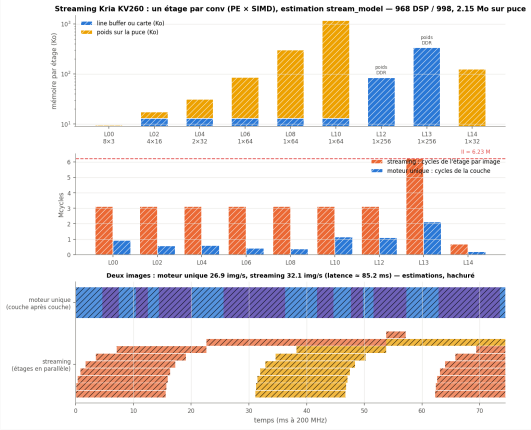
Formats on the same model (M9.2, M9.3)

Weights / activations	Method	mAP	Gap to INT8
INT8 per channel (reference)	PTQ	55.66	—
uniform6 (± 31)	PTQ	54.35	-1.31
mixed6 (powers of 2)	PTQ, direct projection	52.46	-3.20
mixed6	ADMM, 600 iterations, then projection	43.07	-12.59
w4a4	PTQ (power-of-2 steps)	17.06	-38.60
w4a4	QAT, 600 iterations	36.38	-19.28
INT8 + hardware post-processing	unsorted NMS, Q4 boxes	55.56	-0.10



REQ-YOLO: weights as sums of two powers of two, no DSP for MACs; projection 52.46, ADMM *did not converge* (43.07).
W4A4: PTQ 17.06 $\rightarrow$ short QAT 36.38 (loss still decreasing). Both exports are bit-exact in golden and C-sim.

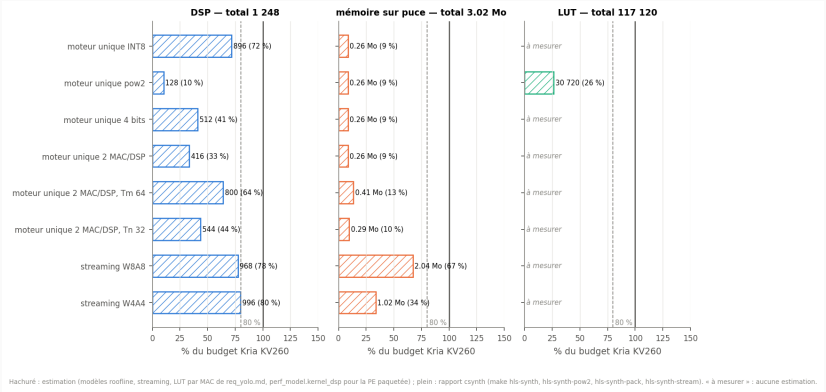
Streaming architecture (M9.4)



One stage per convolution with line buffers. The weights of Tiny-YOLOv2 do not fit on chip: a **hybrid** plan, the two largest layers read theirs from DDR once per image.

Estimates: 32.1$^{\text{est.}}$ img/s with 968$^{\text{est.}}$ DSP, latency 85.2$^{\text{est.}}$ ms; 64.2$^{\text{est.}}$ img/s with 4-bit weights. The streaming kernel equals the golden model stage by stage in C-sim.

Resources: single engine vs streaming



Roofline model for the single engine (896 DSP, 112 BRAM18) against the streaming plan. Real resources will only be known after synthesis.

Dependability (M25)

Faults and an exact checksum

The bit-exact integer model is a fault-free reference: any difference is caused by the injected fault.

- **Injector**: bit flips (given bit or error rate) and stuck-at in INT8 weights, int32 biases, INT8 activations, int32 accumulator; seeded; no fault = integer model exactly
- **Campaign**: mAP against error rate, layer and bit position; smoke run only, full grids to run
- **ABFT**: sum of the output channels = conv of the input by the summed filters. All integer before requantization $\Rightarrow$ **exact modulo** 2^{32}, no tolerance, no false alarm
- in the integer model and the golden model: 0 errors on the dumps, every accumulator fault detected; HLS lane pending synthesis

Out-of-distribution monitor

Reads the INT8 output of the last backbone layer of Tiny-YOLOv2 (already written by the accelerator), scored on the ARM.

- dark and thermal images give *weaker* features: they fall inside the in-domain shell, so raw distances are *lower* out of domain
- the monitor uses the **typicality** of the score: distance to the median on held-out VOC images, in either direction
- the plain feature norm wins: AUROC 0.74 (low light), **0.88** (thermal), for a negligible share of the MACs

Score	AUROC ExDark	AUROC FLIR	Spearman score / mAP	MACs (share of the detector)
norm	0.744	0.884	−0.66	0.35 M (0.01 %)
Mahalanobis	0.669	0.726	−0.41	4.54 M (0.13 %)
autoencoder	0.686	0.718	−0.14	45.69 M (1.31 %)

Beyond VOC (M11–M18, M21)

Datasets, each chosen to stress one part of the chain

COCO 80 classes, Tiny-YOLOv3 weights

VisDrone aerial, tiny objects

KITTI driving, wide images

FLIR thermal, 1 channel

ExDark low light

CrowdHuman crowds, grid collisions

AU-AIR 1080p frames, class imbalance

DroneVehicle oriented $\rightarrow$ upright boxes

HIT-UAV thermal from 60–130 m

UAVDT 1024×540 video frames

ASL classification $\rightarrow$ detection with MediaPipe hand pseudo-boxes (87.5 % of images get a box)

Per dataset: loader, datasheet, statistics notebook (M16: classes, box sizes, density, anchors, collisions in the grid), archive on Google Drive (rclone) for Colab.

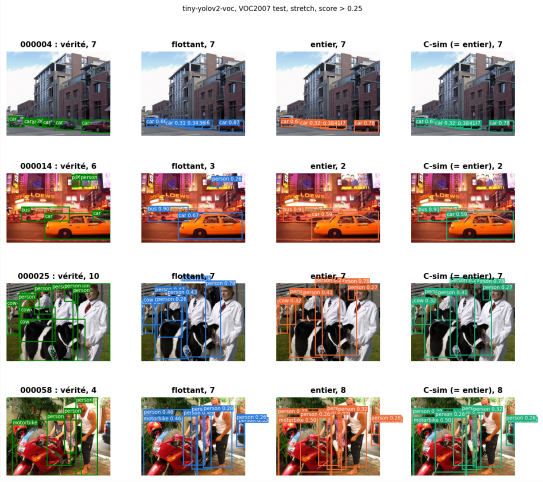
Notebooks and the Colab campaign (M14, M15)

- `make notebooks`: inference, training and sweep (batch × subset) per model × dataset, generated from templates; they only call repository code
- run locally or on a Colab GPU from VS Code; outputs synced to Drive and harvested back (`make harvest`)
- first sweeps: 600 iterations, scored on 50 images (tier R): *ranking*, not published results; none of these runs has converged yet

Best run of the first sweep of each dataset (tier R)

Dataset	Training images	Best run	Batch	Training subset	Metric	Score (50 images)	Status
VOC	16,551	`b32-sall`	32	all	mAP, 11 points	36.27	tier-R
VisDrone	6,471	`b32-sall`	32	all	mAP, 11 points	2.24	tier-R
KITTI	5,985	`b32-sall`	32	all	mAP, 11 points	2.38	tier-R
FLIR	10,742	`b8-sall`	8	all	AP@[.5:.95] (COCO)	0.70	tier-R
ExDark	3,000	`b32-sall`	32	all	mAP, 11 points	13.29	tier-R
CrowdHuman	15,000	`b16-sall`	16	all	mAP, 11 points	35.87	tier-R

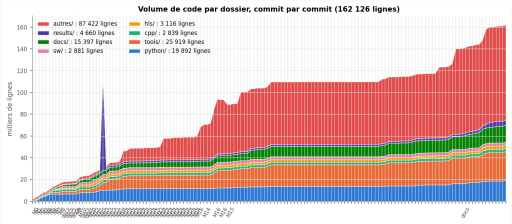
Detections

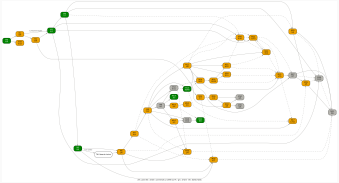

tiny-yolov2-voc, VOC2007 test, stretch, score > 0.25
000004 : vérité, 7
flottant, 7
entier, 7
C-sim (= entier), 7
000014 : vérité, 6
flottant, 3
entier, 2
C-sim (= entier), 2
000025 : vérité, 10
flottant, 7
entier, 7
C-sim (= entier), 7
000058 : vérité, 4
flottant, 7
entier, 8
C-sim (= entier), 8

Tooling and demos (M13, M17, M19–M22)

Everything is regenerated

- **M13**: 88 figures by `make figures` (models, accelerator, results, project, the maths of the reimplementation)
- **M17**: a paper-style article; `make article` collects every number with its status (measured, C-sim, projection, estimate, tier R) — this deck uses the same macros
- **M19**: docs, spec and generated text in English, checked in `make ci`





Live webcam demo (M20)

- `make webcam`: pick a model (float or integer, VOC or COCO), live camera image, continuous inference, boxes and classes on screen
- capture thread at the camera rate (20 FPS measured), inference thread at 3–5 FPS on the CPU ($\approx$ 200 ms): boxes and image always match
- same pipeline as `tools/detect.py`: `preprocess` $\rightarrow$ `detect`/`detect_int` $\rightarrow$ `draw`
- outlook: the same demo on the KV260 with the accelerator

Commands

```
make get-weights
python tools/webcam.py -list
make webcam
python tools/webcam.py \
  -model int:tiny-yolov2-voc
```

Photo auto-labelling (M22)

- `make autolabel`: drag & drop a photo or a folder, classes typed as text
- open-vocabulary labellers: Grounding DINO tiny (default), OWLv2
- manual correction in the browser, export as YOLO txt (one file, or a zip)
- purpose: build training sets for new classes, fine-tune the embedded Tiny-YOLO, run it on the webcam or the board

Idea
Generalises the ASL pseudo-boxes (M21) from hands to any class that can be named. The large labeller never goes on the FPGA.

Conclusion

Limitations and next steps

- **No number is measured on the board yet**: performance is a cycle-model projection, resources are estimates, the FPGA mAP is a C simulation → synthesis, co-sim and board runs (M6–M8), then `run_compare` must show 0 differences
- REQ-YOLO ADMM did not converge; the 4-bit QAT ran for a fraction of an epoch
- NumPy training from scratch (VOC fine-tuning, M2) is not finished: the reference uses Darknet weights
- datasets beyond VOC explored at tier R only: longer runs, matching resize, tiles (M15)
- YOLOv4-tiny: full C-sim pass on COCO and QAT; YOLOv7-tiny accuracy, then YOLOv5n and YOLOv8n (M26)
- fault campaign at full size, ABFT lane in the kernel; safety work planned (M27–M31: metrics, equivalence, runtime monitors, latency bound, assurance case)

Every projected or estimated value carries its status in the slides; T17.19 switches them to measurements once the board runs.

Summary

- A complete detection chain **without a learning library**, every gradient checked
- **Bit-exact** from the NumPy integer model to the C-simulated HLS kernel behind the ARM driver, on the full VOC2007 test set: 56.30 $\rightarrow$ 55.66 $\rightarrow$ $55.66^{\text{C-sim}}$ mAP
- **One engine** for v2, v3 and YOLOv4-tiny, projected at $37.2^{\text{proj.}}$ ms per image on the KV260 ($24.7^{\text{proj.}}$ ms with two MACs per DSP)
- **Dependability**: exact int32 ABFT, fault injection, an OOD monitor almost for free
- INT8, power-of-two, 4-bit and integer post-processing **compared on the same model**
- **Reproducible**: tests, figures, notebooks, article and these slides regenerated by `make`

Questions?